Messaging: the tuning so far, steps 0 to 10

Every step of the tuning order of `doc/messaging-architecture.md` section 6 that has been done, what each one found, what it changed and what it measured. Branch `feat/messaging`, sessions of 28 September to 11 October 2026; the per-step reports in `doc/` hold the detail, this one the whole picture.

1. Where it stands

What a client sees	Baseline, 2026-09-29	Now
A post is acknowledged in (p50, 13 posts/s)	66 ms; 1.5 % lost to a race	53 ms; nothing lost at 13, 40 or 133 posts/s
A post on a conversation with 200 messages	3.5 s, half of them lost	78 ms
The other participant sees it after (p50)	48 ms	45 ms; the Kafka leg is 2 ms of it
The inbox page (20 conversations, four readers)	1.5 s	47 ms; 50 pages/s per node
The read model trails the command side by (p50, 80 posts/s burst)	32 s	20 ms
A client that stops reading	the node kept every frame for it, for good	256 frames, then closed 4408; 0 kept
Two nodes	every event projected twice, every frame delivered twice	one copy; a crashed leader is replaced in 10 s
One statement on the read model	about 30 ms (15 of them a class generated per statement)	4 ms
The command side's ceiling on a laptop	unknown	the store's disk: 30 writes/s; 370/s on tmpfs; no post refused either way

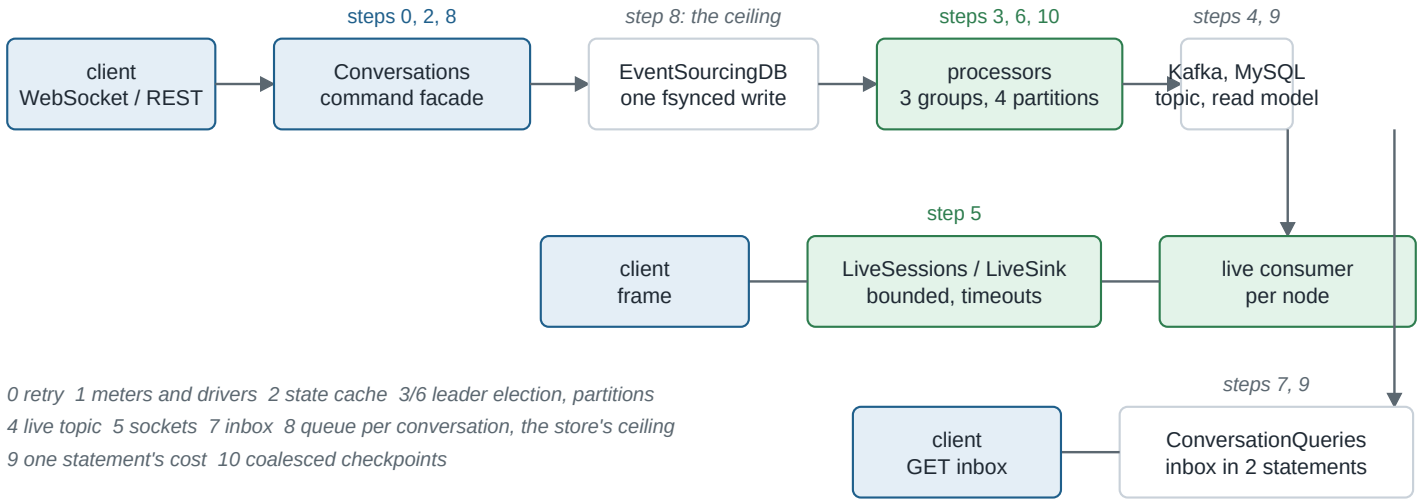

Figure 1. The path of one message and the step that acted on each part of it.

2. The steps

1. Instrumentation (28 to 29 September)

The `messaging.*` meters (command latency and retries, live latency, projection delay and handler time, sockets, outbound buffers, later query latency and transport), p50/p95/p99 in the process, `ChatLoadDriver` for the agreed load shape and a baseline. Two defects found by it: a read-your-writes gap on the socket handshake and concurrent posts lost to the store's optimistic concurrency.

0. The two defects (1 October)

The handshake re-checks a conversation the read model does not have yet; `Conversations.send` re-dispatches a command the store refused. Rerun on 10 October: 1,599 of 1,599 posts acknowledged, 361 retries absorbed.

2. State cache (10 October)

`opencqrs.command-handling.cache.type=in_memory`, 1,000 conversations per node: a command replays only the events after the cached state. On 200-message conversations a post went from 3.5 s and half lost to 78 ms and none lost.

3 and 6. Leader election and partitions (10 October)

An R2DBC lock registry in cqrs-reactive and OpenCQRS's leader-election controller: one copy of each group per partition across nodes; four partitions for the read model and a pool of 40. Read-model lag on the 4,800-message burst from 32 s to 0.08 s; a killed leader replaced in 9.8 s with nothing lost.

4. The live topic (11 October)

The Kafka leg measured on its own (`messaging.live.transport`): 2 ms of a 45 ms fan-out, the store's observe pick-up is the rest. Latency settings on the live binding for the production shape and a consumer group named after the node, so restarts leave no dead group.

5. Sockets (11 October)

`SocketLifecycle` on every session: 256 frames then close 4408, a server ping every 30 s, close 4409 after 75 s of silence, a 30 s write timeout. Ten sockets that never read, under 10,000 bulletins: 75,782 frames kept before, none after, the dead sockets gone within 75 s.

7. The inbox (11 October)

The participants of a page in one `IN` query: two statements per page instead of one per row. A page of 20 from 1.5 s to 165 ms with four readers, 1.4 to 20 pages/s, and a timer on every read.

9. The statement cost (11 October)

A layered probe and a Flight Recorder profile: reactive-mybatis-support 3.0.6 generated a ByteBuddy class on every statement, 15 ms of CPU. The library's 3.1.3 fixes it: a mapper call from 19.5 to 4.2 ms, the inbox page to 47 ms, the projection from 39 to 15 ms per event. One version property in db-reactive, every service gets it.

8. The command side (11 October)

The ceiling of the burst was the store's disk (28 ms per fsynced write, 30 commands/s; tmpfs 370/s), and the service amplified it: both participants of a conversation collided on every queued post, 5 to 11 % refused after exhausted retries. `SubjectQueues` runs one command per conversation at a time on a node: no retry, no refusal, 17 % more posts acknowledged at 133 posts/s, nothing changed at 13.

10. The read side at the new ceiling (11 October)

The processor threads spent their time writing the checkpoint, one MySQL round trip per observed event per partition (the events of other partitions included): 330 of 487 s for the read model, 85 of 90 s for the live relay. `R2dbcProgressTracker` now coalesces checkpoints every 250 ms (`cqrs.readside.progress.flush-interval`); at 133 posts/s the read model trails by 20 ms instead of 31 s, the relay by 8 ms instead of 20 s, a post reaches the peer in 368 ms instead of 12 s, and all 7,992 posts of the burst are acknowledged; nothing changes at 13 posts/s.

3. The numbers, baseline to now

One node, the development stack, the same host; the load shape of section 5 of the architecture doc unless stated (20 conversations, 40 sockets, one post per socket every 3 s for 60 s). p50 unless stated.

Metric	Baseline (section 7)	Now	Step
ack latency, driver side	66 ms, 12 of 800 lost	53 ms, 800 of 800	0, 8
ack latency on 200-message conversations	3.5 s, 369 of 798 acked	78 ms, 800 of 800	2
`messaging.live.latency` (event to push)	48 ms	45 ms	4
`messaging.live.transport` (the Kafka leg)	not measured	2 ms	4
`messaging.projection.handler`, read model, per event	41 ms	15 ms	9
`messaging.projection.delay`, read model, 4,800-event burst	32 s	0.08 s (3/6); 20 ms at 133 posts/s (10)	3, 6, 10
inbox page of 20, four readers	1.5 s	47 ms	7, 9
one mapper statement	about 30 ms	4 ms	9
frames kept for ten dead sockets after a 10,000-bulletin storm	75,782 (about 80 MB)	0	5
posts refused at 40 posts/s on the disk store	131 of 2,400	0	8
posts acknowledged at 133 posts/s on a tmpfs store	6,290 of 7,986, 842 refused	7,390 of 7,986, 0 refused	8
read-model lag at 133 posts/s on tmpfs	31 s (checkpoint per event)	20 ms	10

4. What was learned on the way

- **Measure the layer, not the system.** Three of the ten steps ended somewhere else than where they started: the fan-out latency was the store's pick-up, not Kafka; the statement cost was a class generated per statement, not MySQL; the command-side ceiling was the store's disk, not the service.
- **Two kinds of profile.** A Flight Recorder run under surefire's `argLine` found the ByteBuddy generation (CPU samples); the same under `spring-boot.run.jvmArguments` with `jdk.ThreadPark#threshold=1ms` found the checkpoint write (where the processor threads park).
- **Load rules.** A fresh store for every run (clearing checkpoints against a non-empty store replays everything); a tmpfs store of 4 GB for runs about the service (1 GB makes the store read-only after 7,500 events; the evaluation licence caps a store at 25,000); drivers on 127.0.0.1 (the host maps

`localhost` to a stale LAN address too); the multi-table DELETE of the clean-up needs the database selected.

- **Reactor Netty.** Cancelling a WebSocket inbound makes it send an empty close frame of its own (clients see 1005): close with the session, never with a timeout on the inbound. Counts come from the sink, not from meter differences.
- **Shared `~/.m2`.** Another worktree installing an older db-reactive or cqrs-reactive puts the old library back for everyone; `ReactiveMybatisProxyTest` and a failing test compile are the tells.
- **The store.** Durable writes on a laptop disk cost 28 ms each and the store takes them one at a time; nothing in a client changes that. Measure on the production storage before any further command-side work.

5. What remains

- **Direct local push** of live frames from the command side to the sockets of the posting node (the follow-up of step 4): removes the 40 ms store pick-up for those sockets, needs duplicate suppression by message id.
- **Relay partitions** if a relay ever lags once the checkpoint is off its path; the live relay handles an event in 0.6 ms, so that is far off.
- **A bound on commands in flight** per node with a quick 503, in front of the facade, if the queue in front of a slow store ever needs one (step 8's note).
- **The production shape** of the store: an NVMe or a storage layer with a fast fsync path moves the command-side ceiling by an order of magnitude; the numbers here are a laptop's.
- **The cross-node race** stays covered by the retry; `messaging.command.retries` should read near zero on one node and small on several.

6. Commits and reports

Commit	Step	Report
3e19a179	1 instrumentation and baseline	`doc/messaging-architecture-guide.pdf`
1ecc4acb	0 the two defects	
1e5bd516, 954c4b16	2 state cache; load rerun	`doc/messaging-tuning-report.pdf` (10 October)
6a52c76c	3 and 6 leader election and partitions	`doc/messaging-leader-election-report.pdf`
c2fffcc6	4 and 5 live topic and sockets	`doc/messaging-sockets-report.pdf`
5af5d914	7 inbox	`doc/messaging-inbox-report.pdf`
72db52ac, 3d10dd89	9 statement cost	`doc/messaging-statement-cost-report.pdf`
a68d0914	8 command side	`doc/messaging-command-side-report.pdf`
uncommitted	10 read side	this report, section 7h of the architecture doc

Branch state: step 10 (the tracker, its test, the docs and this report) is uncommitted on feat/messaging on top of the step 8 commit a68d0914; the branch is not merged into announce-jo-branch and not pushed.